Cypress Interview Questions and Answers

Basic Cypress Questions

Q1: What is Cypress?
A: Cypress is a JavaScript-based end-to-end testing framework that is used for automating web application testing. It works directly in the browser and allows for fast and reliable testing of web apps.

Q2: How does Cypress differ from Selenium?
A:

- **Selenium** is a web automation framework that interacts with browsers using WebDriver and requires external drivers.
- **Cypress** runs directly in the browser, and it interacts with the DOM and JavaScript, offering a faster, more reliable execution.

Q3: What are the key features of Cypress?
A:

- Real-time browser interaction
- Automatic waiting for elements
- Spies, stubs, and clocks for testing
- Network traffic control and assertions
- Screenshots and video recording on test failure
- Easy setup and configuration

Q4: How do you install Cypress in a project?
A: You can install Cypress via npm by running:

```
npm install cypress --save-dev
```

Q5: How do you run Cypress tests?
A: You can run Cypress tests via the command line by running:

```
npx cypress open
```

or for headless execution:

```
npx cypress run
```

Intermediate Cypress Questions

Q6: How do you create a Cypress test?
A: Cypress tests are created in JavaScript files inside the cypress/integration folder. You write tests using Mocha syntax (describe, it). Example:

```
describe('My first test', () => {
  it('should visit a webpage and check the title', () => {
```

```
  cy.visit('https://example.com');
  cy.title().should('include', 'Example');
 });
});
```

Q7: How can you interact with elements in Cypress?
A: You can interact with elements using commands like cy.get(), cy.click(), cy.type(), etc. Example:

```
cy.get('.login-button').click();
cy.get('#username').type('user');
cy.get('#password').type('password123');
cy.get('.submit-button').click();
```

Q8: What are Cypress commands and how do they work?
A: Cypress commands are functions like cy.visit(), cy.get(), cy.click(), cy.type(), and many others. They interact with the DOM and return objects that you can chain further commands to. Cypress commands are asynchronous, but Cypress automatically waits for the elements to appear before performing actions.

Q9: What is the cy.wait() command used for?
A: The cy.wait() command is used to pause the test for a specified amount of time or wait for a network request to finish before continuing with the test.

```
cy.wait(2000); // Wait for 2 seconds
cy.wait('@getUserData'); // Wait for an alias to finish
```

Q10: How do you assert the visibility of an element in Cypress?
A: You can use the .should() assertion to check for visibility, like so:

```
cy.get('.my-element').should('be.visible');
```

Advanced Cypress Questions

Q11: How do you handle network requests and responses in Cypress?
A: You can intercept and modify network requests using the cy.intercept() method. For example:

```
cy.intercept('GET', '/api/user', { fixture: 'user.json' }).as('getUserData');
cy.wait('@getUserData');
```

Q12: What is the purpose of cy.fixture()?
A: The cy.fixture() command is used to load external files (e.g., JSON, text, etc.) for use in tests. It's useful for mocking data in tests.

```
cy.fixture('user.json').then((userData) => {
  cy.get('#name').type(userData.name);
});
```

Q13: How do you run tests in parallel with Cypress?
A: Cypress supports parallelization when integrated with a CI/CD pipeline like CircleCI or GitHub Actions. You can enable parallel test execution via the Cypress Dashboard service.

Q14: How do you handle cookies in Cypress?
A: Cypress provides methods to interact with cookies, such as cy.getCookie(), cy.setCookie(), and cy.clearCookie(). Example:

```
cy.setCookie('session_id', '12345');

cy.getCookie('session_id').should('have.property', 'value', '12345');

cy.clearCookie('session_id');
```

Q15: How do you deal with dynamic elements in Cypress?
A: For dynamic elements, you can use cy.get() with the {timeout: 10000} option, or cy.find() to search within a container, and cy.wait() to wait for them to appear.

```
cy.get('.dynamic-element', {timeout: 10000}).should('be.visible');
```

Q16: What are Cypress custom commands?
A: Custom commands in Cypress are user-defined commands that allow you to extend Cypress with reusable functionality. They are defined in the commands.js file inside the support folder.

```
Cypress.Commands.add('login', (username, password) => {
  cy.get('#username').type(username);
  cy.get('#password').type(password);
  cy.get('.submit-button').click();
});
```

Q17: How do you take screenshots in Cypress?
A: Cypress automatically takes screenshots on test failure. You can also manually capture screenshots using the cy.screenshot() method.

```
cy.screenshot('test-failure');
```

Q18: What is the cy.spy() method used for?
A: cy.spy() is used to spy on a function and track its calls, arguments, and results. It's useful for ensuring certain functions are invoked during the test.

```
const callback = cy.spy();

cy.get('.button').click().then(callback);

cy.wrap(callback).should('have.been.calledOnce');
```

Q19: What is the difference between cy.visit() and cy.request()?
A:

- cy.visit() navigates to a new URL and loads the page in the browser.
- cy.request() is used to make HTTP requests directly to a server and is typically used for testing APIs or controlling network interactions without loading the page.

Q20: How do you use Cypress with continuous integration tools like Jenkins or CircleCI?
A: You can integrate Cypress with CI tools like Jenkins or CircleCI by adding the appropriate scripts to run Cypress tests in your configuration file (e.g., circle.yml or Jenkinsfile). The Cypress Dashboard service can provide insights and reports from CI runs.

JavaScript-Specific Cypress Questions

Q21: How do you pass dynamic values from JavaScript into Cypress commands?
A: You can use JavaScript variables in Cypress commands by using .then() to pass the dynamic data between commands.

```
let username = 'user1';

cy.get('#username').type(username);
```

Q22: How do you deal with asynchronous behavior in Cypress tests?
A: Cypress automatically waits for commands to complete before moving on to the next one, making it easier to work with asynchronous behavior without needing to manage Promises or callbacks explicitly.

Q23: Can you use JavaScript's async/await with Cypress commands?
A: No, Cypress commands are not compatible with async/await because they are asynchronous by nature and return chainable commands, but Cypress manages their execution automatically. Instead, use .then() to handle asynchronous values.

Q24: How can you access data from a network request in Cypress?
A: You can use cy.intercept() to intercept network requests and responses and access the response data.

```
cy.intercept('GET', '/api/user').as('getUserData');

cy.wait('@getUserData').then((interception) => {

  const userData = interception.response.body;

  cy.get('#username').type(userData.name);

});
```

Q25: How do you perform data-driven testing in Cypress?
A: Data-driven testing in Cypress can be done by iterating over an array of test data using cy.each() or a for loop.

```
const testData = [{user: 'user1', pass: 'password1'}, {user: 'user2', pass: 'password2'}];

testData.forEach(data => {

  it('should log in with valid credentials', () => {

    cy.get('#username').type(data.user);

    cy.get('#password').type(data.pass);

    cy.get('.submit-button').click();
```

```
  });
});
```

Reporting and Assertions

Q26: How do you handle assertions in Cypress?
A: Cypress uses the Chai assertion library for making assertions. You can assert the state of elements using .should() or .and(). Examples:

```
cy.get('.login-form').should('be.visible');

cy.get('.submit-button').should('have.text', 'Submit');
```

Q27: What is the difference between .should() and .and() in Cypress?
A:

- .should() is used to assert an expectation on a given element.
- .and() is used to chain multiple assertions on the same element. Example:

```
cy.get('.login-form')
  .should('be.visible')
  .and('have.class', 'form-active');
```

Q28: How do you use .contains() in Cypress?
A: The .contains() command is used to search for elements containing specific text, which is useful for finding elements dynamically.

```
cy.contains('Submit').click();
```

Q29: How do you handle custom assertions in Cypress?
A: You can create custom assertions using the chai or expect API. For example:

```
chai.use(function (_chai, utils) {
  _chai.Assertion.addMethod('haveClassAndText', function (className, text) {
    const element = this._obj;
    utils.flag(this, 'args', [className, text]);
    this.assert(
      element.hasClass(className) && element.text() === text,
      'expected element to have class #{exp} and text #{exp}',
      'expected element not to have class #{exp} and text #{exp}',
      className,
      text
    );
```

```
  });
});
```

Q30: How do you handle waiting for elements in Cypress?
A: Cypress automatically waits for elements to appear before performing actions, but you can also use .wait() to introduce explicit waits or .should() to assert visibility after waiting.

```
cy.get('.dynamic-element').should('be.visible'); // Implicit wait

cy.wait(5000); // Explicit wait
```

Q31: What is the cy.should('exist') assertion used for?
A: The cy.should('exist') assertion ensures that an element exists in the DOM, whether it is visible or not.

```
cy.get('.login-form').should('exist');
```

Command-Line Execution

Q32: How do you run Cypress tests from the command line in headless mode?
A: You can execute Cypress tests from the command line in headless mode using:

```
npx cypress run
```

This will run the tests in the headless Electron browser by default.

Q33: How do you specify a different browser for Cypress in command-line execution?
A: You can specify a browser when running tests from the command line using the --browser flag:

```
npx cypress run --browser chrome
```

Q34: How do you run a specific test file from the command line?
A: You can specify a test file to run using the --spec flag:

```
npx cypress run --spec 'cypress/integration/myTest.spec.js'
```

Q35: How do you pass environment variables in Cypress command-line execution?
A: You can pass environment variables using the --env flag:

```
npx cypress run --env user=username,pass=password
```

You can then access these values in your tests with Cypress.env('user') and Cypress.env('pass').

Q36: How do you generate a test report when running Cypress from the command line?
A: You can generate a test report using plugins like mochawesome. First, install the plugin:

```
npm install mochawesome --save-dev
```

Then configure Cypress to generate a report:

```
{
  "reporter": "mochawesome",
```

```
  "reporterOptions": {
    "reportDir": "cypress/reports",
    "overwrite": false,
    "html": true,
    "json": true
  }
}
```

Video and Recording Features in Cypress

Q37: How do you enable video recording for Cypress tests?
A: Cypress automatically records videos for test runs in the headless mode. You can enable or disable this in cypress.json:

```
{
  "video": true
}
```

Q38: Where are the video files stored in Cypress?
A: By default, Cypress saves video recordings in the cypress/videos folder. The video file name will be based on the test spec and the run.

Q39: How do you disable video recording in Cypress?
A: You can disable video recording by setting "video": false in cypress.json:

```
{
  "video": false
}
```

Q40: How do you upload Cypress videos to a continuous integration service?
A: You can upload videos to a CI service like Jenkins or CircleCI by storing the cypress/videos folder as an artifact. For CircleCI, this can be configured in the .circleci/config.yml file:

```
- store_artifacts:
    path: cypress/videos
```

Q41: How do you take screenshots during test execution in Cypress?
A: Cypress automatically captures screenshots when a test fails. You can also capture screenshots manually using cy.screenshot():

```
cy.screenshot('my-screenshot');
```

Q42: How do you specify screenshot options in Cypress?
A: You can configure screenshot options in cypress.json to define where the screenshots are stored, whether to capture them on command failure, etc. Example:

```
{
  "screenshotOnRunFailure": true,
  "screenshotsFolder": "cypress/screenshots"
}
```

Q43: How do you handle videos and screenshots in Continuous Integration (CI) environments?
A: In CI environments, you can store videos and screenshots as artifacts, and also configure Cypress to upload these artifacts after test completion to monitor the test execution. For example, in Jenkins:

```
post:
  always:
    - archiveArtifacts:
        allowEmpty: true
        artifacts: "cypress/screenshots/*, cypress/videos/*"
```
